

Documento de estudio para la defensa

Fine-Tuning eficiente (LoRA/QLoRA) de LLMs pequeños para análisis de sentimiento en español

Material de preparación — Bloque 1: conceptos densos

Cómo usar este documento

Cada concepto se explica con la **definición rigurosa primero** y la **intuición / analogía como segundo apoyo**, seguidas del **procedimiento**. Después, una caja de **análisis** (qué se decidió/observó, por qué, qué se descartó y, obligatoriamente, qué significa y *qué NO significa*) y un **autotest**. En los ítems difíciles el autotest termina con una *derivación en blanco*: con solo las dimensiones de entrada, reconstruir la fórmula y la cuenta de parámetros de memoria.

Convenciones de fidelidad a la fuente:

- Todo número está trazado a `paper/main.tex` (sección § o Tabla I–V).
- El conocimiento externo al paper va en rojo: `[externo: ejemplo]`.
- Cualquier número *no* trazable a una tabla/sección va marcado en naranja: `[NO trazable a tabla/sección: ejemplo ilustrativo]`.

1 Cómo leer este proyecto desde cero

El problema, en una frase

Hay millones de tuits en español con opiniones, y queremos que una máquina diga si cada uno es *positivo*, *negativo* o *neutral*. El problema no es solo acertar: es acertar *barato*.

Por qué era caro hasta ahora

Los modelos de lenguaje grandes (LLMs) son como un cerebro con miles de millones de *pesos* (números que codifican lo que el modelo «sabe»). Adaptar ese cerebro a una tarea concreta, a la manera clásica, significa reescribir todos esos pesos (*full fine-tuning*): hace falta una GPU cara y muchos datos etiquetados. Para un estudiante, un autónomo o una pyme, eso es prohibitivo.

La idea que lo cambia: PEFT (LoRA / QLoRA)

En vez de reescribir el cerebro entero, lo **congelamos** y le pegamos unas «notas adhesivas» diminutas que corrigen su comportamiento. Eso es **LoRA**: se entrenan solo unas matrices pequeñas, una fracción mínima del modelo. **QLoRA** va más lejos: además comprime el cerebro congelado a 4 bits para que ocupe mucha menos memoria de GPU. Resultado: se puede afinar un modelo decente en una GPU *gratuita* (el objetivo del proyecto es una T4 de 16 GB).

El reparto de personajes

- **Modelos a afinar**: Qwen3-1.7B y Qwen3-4B (LLMs generativos abiertos, pequeños).
- **Rivales clásicos** (*baselines*): BETO y XLM-RoBERTa (encoders de español, afinados a la antigua), más un suelo léxico (TF-IDF) y una referencia de fábrica (RoBERTuito).
- **Datos**: tuits en español de Cardiff NLP (cortes A y B); e InterTASS 2018, con tuits de España, Costa Rica y Perú (corte C).
- **La regla de medir**: *macro-F1*, una nota que pondera por igual las tres clases.

Las tres preguntas (los tres «cortes») y sus respuestas en una línea

Cut A *¿Cuántos ejemplos hacen falta para que afinar gane a solo «pedírselo» (prompting) al modelo?*

Depende del tamaño: al 1.7B le bastan < 7 ejemplos; el 4B necesita ≈ 50 , porque de partida ya lo hace mejor sin afinar.

Cut B *¿Un modelo pequeño con LoRA gana a los encoders clásicos teniendo en cuenta el coste?*

Sí. LoRA «domina la frontera»: más calidad (F1 0.707 / 0.698 vs 0.661 de BETO) entrenando una fracción de los parámetros.

Cut C *Un adaptador entrenado en una variedad de español, ¿sirve para otras?*

Casi siempre sí, pero con una asimetría: entrenar en costarricense o peruano y evaluar en español de España cuesta 0.031–0.039 de F1.

La moraleja

El proyecto no vende «un modelo mejor», sino **evidencia reproducible** sobre *cuándo* y *a qué coste* conviene cada opción, en hardware al alcance de cualquiera. Por eso muchos resultados que parecen «negativos» (una penalización aquí, QLoRA más lento allá) son en realidad la prueba de que el experimento estaba bien diseñado: midió lo que decía medir.

2 LoRA: adaptar un modelo gigante moviendo casi nada

Definición rigurosa

Sea $W_0 \in \mathbb{R}^{d \times k}$ una matriz de pesos preentrenada y **congelada**. LoRA reparametriza su actualización como un producto de *rango bajo*:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k).$$

La capa, en lugar de $h = W_0 x$, calcula

$$h = W_0 x + \frac{\alpha}{r} B A x.$$

Solo A y B se entrenan; W_0 permanece fijo. El factor α/r escala la contribución del adaptador. En el proyecto: $r = 16$, $\alpha = 32$ (luego $\alpha/r = 2$), *dropout* 0,05, sobre las proyecciones de atención {q,k,v,o}_proj.

Fuente: §III-C «LoRA Configuration» de *paper/main.tex*

La intuición (apoyo)

Un LLM es como una **enciclopedia enorme** ya impresa que no queremos reimprimir. LoRA la deja **congelada** y le añade una **hoja de erratas muy fina**: «donde pone esto, entiende esto otro». Esa hoja (las matrices A, B) es pequeñísima porque muchos ajustes útiles «cabén» en pocas dimensiones; el número de dimensiones es el rango r (aquí 16): más rango = corrección más detallada, pero más cara.

El procedimiento

1. Eliges qué matrices corregir: aquí {q,k,v,o}_proj (atención).
2. Congelas cada W_0 (no se entrena).
3. Añades $\Delta W = BA$ y entrenas *solo* A, B .
4. Escalas por α/r .
5. Al inferir, ΔW se puede *fundir* en W_0 : en teoría, sin coste extra.

Cuenta de parámetros (el corazón del ahorro). Entrenar W_0 entera cuesta $d \cdot k$ parámetros; LoRA cuesta $r(d+k)$. Con $d = k = 2048$ y $r = 16$:

$$\text{full} = 2048^2 \approx 4,19\text{M}, \quad \text{LoRA} = 16(2048+2048) = 65\,536 \approx 0,066\text{M},$$

es decir $\approx 64\times$ menos *por matriz*. **[NO trazable a tabla/sección: el valor $d = k = 2048$ es ilustrativo; el paper no publica la dimensión oculta de Qwen3]**. Sumando todas las matrices adaptadas, el proyecto entrena **6,42 M** (1.7B) y **11,80 M** (4B) parámetros (*Tabla IV*) sobre modelos de 1,72 B y 4 B pesos totales.

Sobre el « $\sim 10\,000\times$ » (debilidad W14). El factor $10\,000\times$ es del paper original de LoRA [externo: Hu et al., 2021: $r=4$ sobre GPT-3 de 175 B] y así se cita en §II-A. En *este* proyecto, comparando parámetros entrenables, lo real es:

$$\frac{\text{BETO 110M}}{6,42\text{M}} \approx 17\times, \quad \frac{\text{XLM-R 278M}}{6,42\text{M}} \approx 43\times, \quad \frac{\text{full-FT 1,72B}}{6,42\text{M}} \approx 268\times.$$

El argumento de fondo (LoRA entrena una fracción minúscula) es verdadero; solo el número « $10\,000\times$ » no aplica a esta comparación. (*Verificado contra Tablas I y IV.*)

Análisis de la decisión / resultado

Qué Reparametrización de la actualización como producto de rango bajo $\Delta W = BA$, entrenando solo A, B y congelando W_0 .

Por qué Adaptar modelos de miles de millones de pesos entrenando millones; cabe en GPU gratuita y, al poder fundir ΔW , en teoría no añade latencia.

Alternativas descartadas

Full-FT (mueve todo, $\approx 270\times$ más, casi doble VRAM, y aquí no mejora calidad); *adapters clásicos* [externo: Houlby et al., 2019] (añaden latencia); *DoRA/IA³* (*future work*, no evaluados).

Qué significa

Con $r = 16$ sobre la atención basta para superar a encoders full-fine-tuned en esta tarea.

Qué NO significa

No que LoRA iguale a full-FT en *cualquier* tarea; no « $10\,000\times$ » en este setup (W14); y el «sin latencia» es cierto *solo si* se funde el adaptador (la Tabla IV mide 54 vs 33 ms porque se evaluó sin fundir).

Autotest

1. ¿Para qué sirve el factor α/r al variar r ?
2. ¿Por qué la teoría dice «0 latencia» pero la Tabla IV mide 54 ms (LoRA) vs 33 ms (full-FT)?
3. ¿De dónde sale el « $10\,000\times$ » y por qué no aplica a tu Cut B? Da las cifras correctas.
4. **Derivación en blanco.** Dados $d = k = 2048$ y $r = 16$: escribe ΔW con las dimensiones de A y B , deriva el número de parámetros entrenables por matriz con LoRA y con full-FT, y el cociente. (Solución: BA con $B \in \mathbb{R}^{2048 \times 16}$, $A \in \mathbb{R}^{16 \times 2048}$; $\text{LoRA} = 16(2048+2048) = 65\,536$; $\text{full} = 2048^2 = 4\,194\,304$; $\approx 64\times$.)

3 QLoRA: la misma idea, pero el cerebro comprimido a 4 bits

Definición rigurosa

QLoRA carga el modelo base en **4 bits** con la `BitsAndBytesConfig` del proyecto: tipo de cuantización `nf4` (*NormalFloat-4*), doble cuantización activada (`use_double_quant=True`) y `compute_dtype = bfloat16`. Tras cargar, se aplica `prepare_model_for_kbit_training` y se activa *gradient checkpointing*; sobre ese backbone cuantizado se monta la *misma* configuración LoRA ($r = 16$, $\alpha = 32$, etc.).

Fuente: §III-C «QLoRA Configuration», `src/models/lora_finetune.py`

La intuición (apoyo)

Si LoRA congela la enciclopedia, QLoRA además la **imprime en papel finísimo**: cada peso, que ocupaba 16 bits, se guarda con solo 4 bits (16 niveles posibles en vez de miles). El cerebro pesa mucho menos en memoria; el coste es que, para «leerlo», hay que reconstruir cada peso al vuelo (*dequantización*), lo que cuesta tiempo. NF4 es un formato de 4 bits cuyos niveles se reparten suponiendo que los pesos siguen una campana de Gauss [externo: Dettmers et al., 2023], de modo que se pierde menos información que con 4 bits «planos».

Resultados clave en el proyecto (Tabla IV)

- **Memoria:** VRAM pico baja a **4,93 GB** (1.7B) y **6,79 GB** (4B), frente a 8,72 y 16,7 GB de LoRA: reducción de **1,8×** y **2,5×**. El 4B entra así en GPUs de consumo de 8 GB.
- **Calidad:** retiene $0,681/0,698 = \mathbf{97,6\%}$ (1.7B) y $0,701/0,707 = \mathbf{99,2\%}$ (4B) de la F1 de LoRA.
- **Coste:** entrena $\approx \mathbf{2,4\times}$ más lento (409 vs 169 s; 534 vs 218 s) y la latencia de inferencia $\approx \mathbf{2\times}$ (102 vs 54 ms; 132 vs 68 ms), por la dequantización al vuelo.

Análisis de la decisión / resultado

Qué Cuantización NF4 a 4 bits del backbone congelado + doble cuantización, con LoRA encima.

Por qué Reducir la VRAM pico para meter el 4B en hardware de consumo (8 GB), el objetivo «free-tier» del proyecto.

Alternativas descartadas

LoRA sin cuantizar (el 4B no baja de 16 GB); cuantización de 8 bits (menos ahorro); 4 bits «planos» en vez de NF4 (peor preservación de la información).

Qué significa

A esta escala, cuantizar a 4 bits cuesta *casi nada* en calidad (1–2,5 puntos de F1) y mucho en VRAM. La elección LoRA-vs-QLoRA debe gobernarse por el eje de coste (VRAM/latencia), no por la calidad esperada.

Qué NO significa

No que QLoRA sea «gratis»: paga 2,4× en entrenamiento y 2× en latencia. Para inferencia de baja latencia (< 100 ms), LoRA sin cuantizar es mejor si la VRAM lo permite.

Autotest

1. ¿Por qué QLoRA es más *lento* aunque ocupe menos memoria?
2. ¿Qué aporta NF4 frente a 4 bits «planos»?

3. Para el 4B, ¿cuál es la retención de F1 y la reducción de VRAM? ¿Qué implica para elegir entre LoRA y QLoRA?
4. **Derivación en blanco.** A partir de las cifras de la Tabla IV, calcula el factor de reducción de VRAM de QLoRA vs LoRA para 1.7B y 4B, y el porcentaje de F1 retenido en cada caso. (Solución: $8,72/4,93 \approx 1,8\times$ y $16,7/6,79 \approx 2,5\times$; 97,6% y 99,2%.)

4 Macro-F1: la regla con la que se mide todo

Definición rigurosa

Para una clase c , con TP_c, FP_c, FN_c (verdaderos positivos, falsos positivos, falsos negativos):

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2 P_c R_c}{P_c + R_c} \text{ (media armónica).}$$

El **macro-F1** promedia sin pesos sobre las $C = 3$ clases:

$$\text{macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c = \frac{F1_{\text{neg}} + F1_{\text{neu}} + F1_{\text{pos}}}{3}.$$

Fuente: §III-D y §IV de *paper/main.tex*

La intuición (apoyo)

Es un examen con **tres temas de igual peso**: tu nota es la *media de las tres notas*, no el total de aciertos. Si fuera el total, podrías aprobar dominando los dos temas fáciles e ignorando el difícil. Aquí el tema difícil es **neutral** (detectar que un tuit *no* expresa emoción). *Accuracy* es «total de aciertos» y puede salir alta ignorando neutral; macro-F1 obliga a no abandonarlo. La media armónica es alta solo si *precisión y recall* lo son: si $P = 0,9$ y $R = 0,1$, $F1 = 2 \cdot \frac{0,9 \cdot 0,1}{1,0} = 0,18$ (no 0,5).

Anclaje en los datos

En la Tabla V, XLM-RoBERTa logra $F1_{\text{neu}} = 0,500$ (el paper lo llama «equivalente a asignación aleatoria»), BETO 0,550, mientras pos/neg rondan 0,70–0,74. LoRA cierra ese hueco (0,613 el 4B, 0,631 el 1.7B). Como macro-F1 da igual peso a la clase difícil, premia ese cierre: por eso LoRA «gana» en macro-F1 aunque empate en las clases fáciles.

Análisis de la decisión / resultado

Qué	Media no ponderada de la F1 de las tres clases; métrica primaria del estudio.
Por qué	El test está balanceado (290/clase) pero la <i>dificultad</i> no; macro-F1 trata las clases por igual y refleja el rendimiento en la clase difícil.

Alternativas descartadas

Accuracy/micro-F1 (se dejan dominar por el agregado y ocultan el fallo en neutral); *weighted-F1* (pondera por frecuencia; con clases balanceadas apenas cambia y no penaliza la clase difícil).

Qué significa

Macro-F1 alto = rendimiento *equilibrado* en las tres clases.

Qué NO significa

No mide calibración; no distingue *tipos* de error (pos↔neu vs pos↔neg); y **no es comparable entre datasets con distinto número de clases**.

Autotest

1. Define con tus palabras precisión y recall para la clase neutral.
2. ¿Por qué F1 usa media armónica y no aritmética? Da un ejemplo numérico.
3. Un modelo que predice *siempre* «positivo» en test balanceado de 3 clases: ¿qué accuracy y qué macro-F1 aproximado tiene? (Pista: $\text{acc} = 1/3$; solo pos con recall 1 y precisión $\approx 1/3 \Rightarrow F1_{\text{pos}} \approx 0,5$, las otras 0 $\Rightarrow \text{macro} \approx 0,17$.)
4. ¿Por qué el paper la llama «la cifra más informativa»? Relaciónalo con la Tabla V.

5 El objetivo SFT: solo la etiqueta cuenta

Definición rigurosa

El *fine-tuning* es supervisado (SFT) con *cross-entropy*, pero **enmascarando el prompt**: a todos los tokens del prompt se les asigna la etiqueta -100 (valor que la función de pérdida ignora), de modo que solo el **token de etiqueta** (una de *negative*, *neutral*, *positive*) y el token EOS contribuyen a la pérdida. Formalmente, si $\mathcal{M}$ es el conjunto de posiciones no enmascaradas,

$$\mathcal{L} = -\sum_{t \in \mathcal{M}} \log p_{\theta}(y_t | y_{<t}), \quad \mathcal{M} = \{\text{token de etiqueta, EOS}\}.$$

La estructura del prompt (system prompt, chat template, *thinking mode* off) es *idéntica* a la del baseline de prompting.

Fuente: §III-C «Model Selection and SFT Objective»; *configs/prompting_protocol.yaml*

La intuición (apoyo)

No queremos enseñar al modelo a *recitar la pregunta*, solo a *dar la respuesta correcta*. Por eso «tapamos» (máscara -100) todo el enunciado y solo puntuamos la palabra final (la etiqueta). Es como un examen donde solo corrigen la casilla de la respuesta, no si copiaste bien el enunciado.

Por qué importa este detalle

Como el prompt es idéntico entre prompting y fine-tuning, y solo la etiqueta entra en la pérdida, **la única variable que cambia entre «pedírselo» y «afinarlo» son los pesos actualizados**. Eso es lo que hace *limpio* el experimento de Cut A: el cruce prompting→fine-tuning no se puede atribuir a diferencias de protocolo.

Análisis de la decisión / resultado

Qué Cross-entropy con enmascarado del prompt (-100); solo etiqueta + EOS contribuyen.

Por qué Aislar la variable «pesos» entre prompting y fine-tuning, y centrar el aprendizaje en producir la etiqueta correcta.

Alternativas descartadas

Pérdida sobre *todos* los tokens (mezclaría aprender la tarea con memorizar el prompt) o cabezal de clasificación dedicado (rompería la equivalencia con el harness generativo del baseline).

Qué significa

Prompting y fine-tuning comparten todo salvo los pesos; el experimento de cruce es justo.

Qué NO significa

No que el modelo aprenda «formato» (eso ya lo da el prompt); no que se entrene un clasificador clásico: sigue siendo generación, solo que supervisada en el token de etiqueta.

Autotest

1. ¿Qué representa el valor -100 y sobre qué tokens se aplica?
2. ¿Por qué es importante que el prompt sea idéntico al del baseline de prompting?
3. **Derivación en blanco.** Escribe la forma de $\mathcal{L}$ indicando explícitamente qué posiciones componen $\mathcal{M}$ y por qué el resto no suma a la pérdida.

6 Caracterización en sigmas: magnitud, no significancia

Definición rigurosa

En Cut C, cada celda (variedad origen $\rightarrow$ variedad destino) se evalúa con 3 semillas {42, 43, 44}; se reporta media $\pm$ desviación típica (std) entre semillas. Un «gap» Δ entre dos celdas se expresa en **unidades de std de semilla**:

$$\text{gap en } \sigma = \frac{|\Delta|}{\sigma_{\text{semilla}}}.$$

Ejemplo (Qwen3-1.7B, CR $\rightarrow$ ES): ES en-dominio $0,644 \pm 0,007$, CR $\rightarrow$ ES $0,613 \pm 0,007$, luego $\Delta = -0,031$, y $0,031/0,007 \approx 4,4\sigma$. **[NO trazable a tabla/sección: el paper reporta $4,6\sigma$ para esta celda; recalculando con la σ redondeada de la tabla sale $\approx 4,4\sigma$ (diferencia por redondeo de σ antes de dividir)].**

Fuente: §IV-C de *paper/main.tex*; Tabla III (*tab:cutC*)

La intuición (apoyo)

« $4,6\sigma$ » suena a estadística, pero aquí significa solo: *la caída es 4,6 veces el ruido que hay entre semillas*. Es una forma de decir «esto es grande comparado con lo que varía al cambiar la semilla aleatoria», no «esto es significativo» en el sentido de un test de hipótesis.

El resultado y su honestidad

La penalización al evaluar en ES (entrenando en CR/PE) es $0,031\text{--}0,039$ de F1 ($2,6\text{--}4,6\sigma$), y **replica en 1.7B y 4B**. El paper avisa explícitamente: no hay test de hipótesis (permutación, t de Student), ni corrección por comparaciones múltiples sobre la matriz $3 \times 3 \times 2$; el criterio de 2σ es un *heurístico de magnitud*, no un p -valor.

Análisis de la decisión / resultado

Qué	Expresar los gaps de transferencia en múltiplos de la std entre semillas.
Por qué	Con corpus pequeños (539–738 ejemplos) y 3 semillas, da una medida <i>honest</i> de magnitud relativa al ruido, sin sobrevender significancia estadística.

Alternativas descartadas

Test formal con corrección de comparaciones múltiples (queda como *future work*; con 3 semillas tendría poca potencia); reportar solo Δ sin contexto de ruido (no diría si es grande o pequeño).

Qué significa

La asimetría hacia ES es *grande respecto al ruido y reproducible* en ambos tamaños de modelo.

Qué NO significa

No es significancia estadística (p -valor); no prueba causalidad dialectal; con más semillas/datos podría matizarse. La excepción CR→PE ($3,9\sigma$ en 1.7B) *no* replica en 4B ($0,3\sigma$): solo la columna ES es robusta.

Autotest

1. ¿Qué afirma y qué *no* afirma un «gap de $4,6\sigma$ » aquí?
2. ¿Por qué el paper no aplica un test de hipótesis formal? ¿Qué tendría que cambiar para hacerlo?
3. **Derivación en blanco.** Dados ES en-dominio $0,644 \pm 0,007$ y PE→ES $0,613 \pm 0,008$, calcula Δ y el gap en σ . (Solución: $\Delta = -0,031$; $0,031/0,008 \approx 3,9\sigma$, coherente con el $3,8\sigma$ del paper.)

7 El punto de cruce y su mecanismo

Definición rigurosa

El **punto de cruce** es el menor n (tamaño del set de entrenamiento) para el que LoRA supera el *mejor* resultado de prompting del *mismo* modelo (N-independiente). De la Tabla II:

- Qwen3-1.7B: mejor prompting $0,560$ ($k=8$). LoRA está por debajo en $n=4$ ($0,539 \pm 0,010$) y **lo supera en** $n=7$ ($0,590 \pm 0,063$).
- Qwen3-4B: 0-shot ya $0,633$; mejor prompting $0,652$ ($k=4$). En $n=7$ ($0,645 \pm 0,038$) y $n=25$ ($0,645 \pm 0,037$) sigue por debajo; **crucza hacia** $n \approx 50$ ($0,666 \pm 0,034$).

Fuente: §IV-A y Tabla II (*tab:cutA*); mecanismo en §II-C y §V

La intuición (apoyo) y el mecanismo

«Pedírselo» (prompting) tiene un **suelo de calidad** que depende de lo que el modelo ya sabe de fábrica. Las demostraciones del prompt sobre todo comunican *formato* y *espacio de etiquetas*, no enseñan el mapeo correcto [externo: Min et al., 2022]. Por eso un modelo más grande (4B) parte de un suelo más alto ($0,652$ vs $0,560$) y el fine-tuning necesita *más señal* (más ejemplos) para superarlo. De ahí que el cruce se desplace a la derecha con el tamaño: no hay un umbral universal.

Análisis de la decisión / resultado

Qué El cruce prompting→fine-tuning depende del tamaño: < 7 ejemplos (1.7B) vs ≈ 50 (4B).

Por qué Suelo de prompting $\propto$ conocimiento preentrenado; suelo alto $\Rightarrow$ más datos para superarlo.

Alternativas descartadas

Buscar un umbral único universal (no existe; varía con el suelo del modelo, y probablemente con tarea/dominio).

Qué significa

Regla práctica: con muy pocos labels ($n < 10$) y modelo pequeño, afina ya; con 4B, comprueba primero si el suelo de prompting te basta, porque afinar solo gana fiable hacia $n \approx 50$.

Qué NO significa

Los números ($n < 7$, $n \approx 50$) *no* son constantes universales; lo reproducible es la *existencia* del desplazamiento con el tamaño.

Autotest

1. ¿Por qué un modelo más grande *retrasa* el punto de cruce?
2. ¿Qué papel juegan las demostraciones del prompt según Min et al.?
3. ¿Qué parte de este hallazgo es universal y qué parte depende del modelo?

8 Muestreo estratificado y disjunto por semilla

Definición rigurosa

Para Cut A se construyen subconjuntos de tamaño $n \in \{4, 7, 10, 16, 25, 50, 100, 250, 500, 1000\}$ más el total (1839). Cada semilla $\{42, 43, 44\}$ extrae un subconjunto **estratificado** (las 3 clases presentes en cada celda; en $n=4$: neg= 2, neu= 1, pos= 1) e **independiente/disjunto** de los de las otras semillas (0 ejemplos compartidos en $n=4$, 0 en $n=10$).

Fuente: §III-A; §IV-A; *scripts/build_fraction_subsets.py*

La intuición (apoyo)

Queremos que la barra de error (la std) signifique algo real. Si las tres semillas vieran *los mismos* ejemplos y solo cambiara la inicialización, la std mediría «ruido de pesos». Al hacer que cada semilla vea ejemplos *distintos*, la std mide algo más informativo: **cuánto depende el resultado de qué datos te tocaron** (varianza de muestreo de datos). La estratificación evita que, por azar, una celda diminuta se quede sin alguna clase.

Análisis de la decisión / resultado

Qué Submuestras estratificadas e independientes por semilla.

Por qué Que la std refleje varianza de *muestreo de datos*, no solo de inicialización; garantizar las 3 clases en celdas diminutas.

Alternativas descartadas

Submuestras solapadas / fijas entre semillas (la std subestimaría la incertidumbre real); muestreo no estratificado (riesgo de clases ausentes en n pequeños).

Qué significa

Las barras de error de Cut A son honestas respecto a la sensibilidad a los datos.

Qué NO significa

No elimina toda incertidumbre: con $n=4$ la varianza es enorme ($\pm 0,089$ en 4B) precisamente porque depende muchísimo de qué 4 ejemplos tocaron.

Autotest

1. ¿Qué mide la std en Cut A gracias a que las submuestras son disjuntas?
2. ¿Por qué se estratifica en lugar de muestrear al azar puro?
3. ¿Por qué la varianza es máxima en $n=4$ y se contrae al crecer n ?

9 Frontera de Pareto y dominación

Definición rigurosa

Un método **domina** a otro si es mejor o igual en *todos* los ejes y estrictamente mejor en al menos uno. La **frontera de Pareto** es el conjunto de métodos no dominados. En Cut B los ejes son calidad (macro-F1) y coste (parámetros entrenables, VRAM, tiempo, latencia). LoRA domina a los encoders: más F1 (0,707/0,698 vs 0,661/0,646) con 6–12 M parámetros entrenables.

Fuente: §IV-B; Tabla IV (tab:cutB); Fig. 2

La intuición (apoyo)

Imagina comprar coche por precio y consumo: un coche está «en la frontera» si no hay otro más barato *y* que gaste menos a la vez. Los que sí tienen a alguien mejor en todo quedan «dominados».

Análisis de la decisión / resultado

Qué LoRA ocupa la frontera calidad-coste; full-FT y encoders quedan dominados (salvo full-FT en latencia).

Por qué Mejor calidad a una fracción del coste de parámetros/VRAM.

Alternativas descartadas

Mirar solo calidad (ocultaría el coste) o solo coste (ocultaría la calidad): la gracia es el *trade-off* conjunto.

Qué significa

Para presupuesto limitado, no hay escenario donde full-FT de un encoder gane a LoRA en los ejes medidos.

Qué NO significa

No que LoRA tenga la mayor F1 absoluta del paper: RoBERTuito saca 0,758, pero está *fuera* de la frontera por ser out-of-domain (TASS 2020, zero-shot) y no comparable de forma controlada (esto es la debilidad W1).

Autotest

1. Define «dominar» con tus palabras y di por qué full-FT queda dominado salvo en latencia.
2. ¿Por qué RoBERTuito (F1 0,758) no está en la frontera pese a tener la F1 más alta?

10 Los cuatro ejes de coste

Definición rigurosa

Por cada celda experimental se miden cuatro costes: (1) **parámetros entrenables** (los que se actualizan, no los totales); (2) **VRAM pico** en GB, vía `torch.cuda.max_memory_allocated()`;

(3) **tiempo de entrenamiento** (wall-clock, s); (4) **latencia** de inferencia (ms/ejemplo, promediada sobre el test).

Fuente: §III-D; Tabla IV

La intuición (apoyo)

«Calidad» a secas engaña: un modelo carísimo que acierta un poco más no siempre compensa. Estos cuatro ejes responden a preguntas reales de despliegue: ¿qué guardo y reentreno por tarea? (params) ¿me cabe en la GPU? (VRAM) ¿cuánto tardo en entrenar? (tiempo) ¿responde rápido? (latencia).

Análisis de la decisión / resultado

Qué	Cuatro dimensiones de coste medidas por celda junto a la calidad.
Por qué	Hacer comparables modelos generativos y encoders en <i>coste</i> , no solo en F1; validar viabilidad en T4 16 GB (la VRAM pico es la cota clave).
Alternativas descartadas	Reportar calidad aislada del coste (lo que hacía la literatura previa, según §II); usar parámetros <i>totales</i> en vez de entrenables (no refleja el coste real de PEFT por tarea).
Qué significa	La distinción entrenables vs totales es la que justifica el ahorro de PEFT (6–12 M frente a backbones de miles de millones).
Qué NO significa	La VRAM se midió en H200, pero es una <i>cota superior</i> válida para T4; los <i>tiempos</i> sí cambiarían en T4 (ver debilidad W4).

Autotest

1. ¿Por qué se miden parámetros *entrenables* y no totales?
2. ¿Cuál de los cuatro ejes es el realmente relevante para la afirmación «free-tier / T4» y por qué?

11 El suelo de pasos y el calendario de entrenamiento

Definición rigurosa

Para que las fracciones diminutas no queden infraentrenadas, el número de pasos de optimización se acota por abajo:

$$\text{max_steps} = \max\left(80, 5 \cdot \left\lceil \frac{n}{8} \right\rceil\right).$$

Calendario común a LoRA/QLoRA: $\text{lr} = 2 \times 10^{-4}$, scheduler *cosine* con warmup ratio 0,1, 5 épocas, batch 8, grad. accum. 1, longitud máxima 256, bf16. La val macro-F1 se evalúa 4 veces durante el entrenamiento.

Fuente: §III-C «Training Schedule»; *configs/sprint3/grid/lora_quen1.7b_n100_s42.yaml*

La intuición (apoyo)

Con muy pocos ejemplos, «5 épocas» son poquísimos pasos: el modelo apenas se entera. El suelo de 80 pasos garantiza un mínimo de práctica aunque solo haya 4 ejemplos. Por encima de cierto n , manda la fórmula $5\lceil n/8 \rceil$ (las 5 épocas reales).

Análisis de la decisión / resultado

Qué Suelo de 80 pasos; resto del calendario fijo.

Por qué Evitar que $n=4, 7, 10$ queden infraentrenados y contaminen la curva de Cut A.

Alternativas descartadas

Fijar «5 épocas» sin suelo (las celdas diminutas casi no se entrenarían) o un n° de pasos fijo igual para todo n (sobreentrenaría los n grandes).

Qué significa

La curva F1-vs- n es comparable entre tamaños porque ninguna celda está infraentrenada por accidente.

Qué NO significa

No que 80 pasos basten para *aprender* con 4 ejemplos: la enorme varianza en $n=4$ muestra que el límite es el dato, no los pasos.

Autotest

1. ¿Qué problema evita el suelo de 80 pasos?
2. **Derivación en blanco.** Calcula `max_steps` para $n=4$ y para $n=1000$. (Solución: $n=4$: $\max(80, 5\lceil 4/8 \rceil) = \max(80, 5) = 80$; $n=1000$: $\max(80, 5\lceil 1000/8 \rceil) = \max(80, 5 \cdot 125) = 625$.)

12 Selección del mejor checkpoint (val macro-F1)

Definición rigurosa

No se usa el *último* paso: se selecciona el checkpoint con la **mayor macro-F1 de validación** (implementado en `BestValF1Callback`). La validación se evalúa 4 veces durante el entrenamiento; la selección de hiperparámetros usa exclusivamente el split de validación (324 ejemplos); el test (870) se toca una sola vez, al final.

Fuente: §III-C «Model Selection»; §III-D

La intuición (apoyo)

Entrenar de más puede empeorar (overfitting). En vez de quedarte con «la última foto», te quedas con «la mejor foto» según un examen de práctica (validación) que no es el examen final (test). Así no contaminas el test ni te quedas con un modelo pasado de cocción.

Análisis de la decisión / resultado

Qué Elegir el checkpoint de mejor val-F1, no el final.

Por qué Evitar overfitting tardío y respetar la separación validación/test (sin fuga).

Alternativas descartadas

Usar el último paso (riesgo de overfitting); seleccionar sobre el test (fuga de datos, invalidaría la comparación).

Qué significa

Las cifras de test son honestas: el test se usó una sola vez.

Qué NO significa

No garantiza el óptimo global; solo el mejor de las 4 evaluaciones de validación realizadas.

Autotest

1. ¿Por qué no quedarse con el último checkpoint?
2. ¿Por qué seleccionar sobre validación y no sobre test?

13 Desbalance de clases y el descarte de NONE (Cut C)

Definición rigurosa

InterTASS 2018 anota 4 polaridades (P, N, NEU, NONE). Se mapean P→positivo, N→negativo, NEU→neutral, y se **descarta NONE** («sin opinión / sin objetivo»). Tras el descarte: 738 ES / 539 CR / 541 PE de entrenamiento; hold-out estratificado del 15% (131/96/96) para selección; evaluación en los dev splits (444/242/262). La clase neutral es más frecuente en PE ($\approx 26\%$) que en ES/CR ($\approx 15\%$).

Fuente: §III-A; §IV-C

La intuición (apoyo)

NONE no es «neutral»: neutral es «tiene postura, pero ni positiva ni negativa»; NONE es «no hay objetivo del que opinar». Mezclarlos sería meter peras en el cajón de las manzanas y *contaminar* el esquema de polaridad. Y como PE tiene casi el doble de neutrales, su distribución es más desbalanceada, lo que explica más varianza y peor F1 en neutral allí.

Análisis de la decisión / resultado

Qué Descartar NONE; trabajar con 3 clases; convivir con desbalance (PE 26% neu).

Por qué NONE es conceptualmente distinto de neutral; fusionarlo contaminaría la polaridad.

Alternativas descartadas

Mapear NONE→neutral (contaminación conceptual); descartar también NEU (perdería una de las 3 clases del estudio).

Qué significa

El esquema de 3 clases de Cut C es coherente con el de Cuts A–B.

Qué NO significa

No elimina el desbalance: PE sigue teniendo neutral sobre-representado, con efectos en su F1 y varianza (limitación declarada).

Autotest

1. ¿Por qué NONE no se mapea a neutral?
2. ¿Qué consecuencia tiene que PE tenga 26% de neutral frente al 15% de ES/CR?

14 Por qué la clase neutral es la difícil

Definición rigurosa

En la Tabla V, el *ranking* por clase es idéntico en *todos* los modelos y métodos: positivo $\geq$ negativo $>$ neutral. Neutral es siempre la más débil (encoders 0,500–0,550; LoRA 0,613–0,631). Que el orden se repita en encoders y generativos indica que la dificultad de neutral es **propiedad de la tarea**, no artefacto de una arquitectura.

Fuente: §IV-B «Per-Class Error Analysis»; Tabla V

La intuición (apoyo)

Positivo y negativo tienen *señal* (palabras de alegría o enfado). Neutral es la *ausencia* de señal afectiva: hay que detectar que *no* hay emoción, lo cual no tiene un patrón superficial distintivo (puede ser lenguaje coloquial sin carga, hechos, u opiniones que se cancelan). Detectar un «no-algo» es intrínsecamente más difícil que detectar un «algo».

Análisis de la decisión / resultado

Qué Neutral es la clase más difícil para todos; LoRA la mejora más que los encoders.

Por qué Neutral = ausencia de señal afectiva, sin patrón superficial.

Alternativas descartadas

Atribuirlo a la arquitectura encoder (descartado: el ranking se repite también en generativos).

Qué significa

La ventaja de LoRA en neutral sugiere que el preentrenamiento más amplio de Qwen aporta señal extra para identificar el «no-sentimiento».

Qué NO significa

Esa explicación causal (Qwen «sabe más» que BETO) es una *hipótesis razonada*, no demostrada (debilidad W9); el dato duro es el +0,06/+0,11 en neutral, no su causa.

Autotest

1. ¿Por qué detectar «neutral» es más difícil que detectar positivo o negativo?
2. ¿Qué evidencia indica que es propiedad de la tarea y no de la arquitectura?

15 La familia PEFT: de dónde viene LoRA y qué hay más allá

Definición rigurosa

PEFT = *Parameter-Efficient Fine-Tuning*: confinar la adaptación a un subconjunto pequeño de parámetros con el backbone congelado. Hitos: **adapters** [externo: Houlby et al., 2019] (capas cuello de botella entre bloques; $< 4\%$ de parámetros, pero añaden latencia); **LoRA** [externo: Hu et al., 2021] (sin latencia al fundir); **QLoRA** [externo: Dettmers et al., 2023] (LoRA + cuantización NF4); **DoRA** [externo: Liu et al., 2022] (descompone la actualización en magnitud y dirección). Todo unificado en la librería **PEFT** [externo: Mangrulkar et al., 2022].

Fuente: §II-A; §VI (Future Work) de *paper/main.tex*

La intuición (apoyo)

Hay muchas formas de «pegar notas adhesivas» a un modelo congelado. Los adapters clásicos añaden piezas nuevas (y por eso pesan al inferir). LoRA logra el mismo efecto sin coste de inferencia (se funde). DoRA e IA³ son primos más recientes que podrían expresar aún más el trade-off calidad-por-parámetro: por eso aparecen como *future work*, no como parte de los experimentos.

Análisis de la decisión / resultado

Qué	Situar LoRA/QLoRA dentro de la familia PEFT y sus alternativas.
Por qué	Justificar la elección (LoRA: sin latencia al fundir; QLoRA: memoria) y delimitar el alcance (DoRA/IA ³ no evaluados).
Alternativas descartadas	Adapters clásicos (latencia en inferencia); DoRA/IA ³ (fuera del alcance, <i>future work</i>).
Qué significa	LoRA/QLoRA son una elección informada dentro de un abanico, no la única opción.
Qué NO significa	No que LoRA sea óptimo entre todos los PEFT; DoRA/IA ³ podrían mejorar la frontera (no testado aquí).

Autotest

1. ¿Qué diferencia a los adapters clásicos de LoRA en *inferencia*?
2. ¿Por qué DoRA e IA³ aparecen como *future work* y no en los experimentos?